Prakash Thapa
D20A- 32

Experiment No.: 6

1. AIM

To create, understand, compile, deploy, and execute a smart contract using Solidity and Remix IDE, and to perform transactions using the Ballot contract by changing the proposal data type from bytes32 to string.

2. COURSE OUTCOMES

- CO3
- CO5
- CO6

3. TOOLS AND LIBRARIES USED

1. Remix IDE
2. Solidity programming language
3. Ethereum Virtual Machine (EVM)
4. Ethers.js library
5. Web3 Type Converter
6. MetaMask or Remix JavaScript VM for deployment and testing

4. THEORY

A **smart contract** is a computer program deployed on a blockchain that executes predefined instructions when specified conditions are satisfied. Smart contracts are commonly written in Solidity for the Ethereum blockchain.

Remix IDE is a browser-based development environment used to write, compile, debug, deploy, and interact with Solidity smart contracts.

The **Ballot contract** is a voting-based smart contract that allows proposals to be created and votes to be cast. It demonstrates concepts such as structures, mappings, functions, access control, and blockchain transactions.

Important Solidity concepts

1. require() statement

The require() statement checks whether a specified condition is true. If the condition is false, execution stops and the transaction's state changes are reverted.

Example:

require(!voters[msg.sender].voted, "Already voted");

This prevents a voter from voting more than once.

2. mapping

A mapping stores key-value associations and is commonly used to track voters, balances, and other data.

Example:

mapping(address => Voter) public voters;

Here, an Ethereum address is associated with a Voter structure.

3. storage

The storage keyword refers to persistent data stored in the contract's blockchain state. Changes to storage made by a successful transaction persist.

4. memory

The memory keyword refers to temporary data that exists only while a function is executing. It is commonly used for temporary strings, arrays, and structures.

5. bytes32 versus string

- bytes32 is a fixed-size byte sequence occupying 32 bytes.
- string is a dynamically sized sequence of characters encoded as UTF-8.
- bytes32 can be efficient for short, fixed-size text such as proposal names.
- string is more convenient for readable text of variable length but can require more storage and gas.

Changing bytes32 to string requires updating the relevant declarations and assignments consistently throughout the contract.

5. TASKS PERFORMED

1. Opened Remix IDE at https://remix.ethereum.org/.
2. Opened the workspace and located the contracts folder.
3. Opened and studied the Ballot.sol smart contract.

4. Examined the constructor, proposal structure, voter mapping, and voting functions.
5. Modified the proposal name data type from bytes32 to string.
6. Updated the relevant declarations and assignments to support the new data type.
7. Compiled the modified Solidity program using the Solidity Compiler in Remix IDE.
8. Selected an appropriate deployment environment, such as Remix VM.
9. Deployed the compiled Ballot contract.
10. Interacted with the deployed contract to inspect proposals and test the voting functions.
11. Observed the transaction results and verified the contract's behavior.

6. PROGRAM

The following is a simplified Ballot smart contract demonstrating proposal names using the string data type. It is suitable for compiling and testing in Remix IDE.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Ballot {
  struct Voter {
    uint weight;
    bool voted;
    uint vote;
  }

  struct Proposal {
    string name;
    uint voteCount;
  }

  address public chairperson;

  mapping(address => Voter) public voters;

  Proposal[] public proposals;

  constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
```

```
      proposals.push(
        Proposal({
          name: proposalNames[i],
          voteCount: 0
        })
      );
    }
}

function giveRightToVote(address voter) public {
    require(
      msg.sender == chairperson,
      "Only chairperson can give right to vote"
    );

    require(!voters[voter].voted, "Voter has already voted");
    require(voters[voter].weight == 0, "Voter already registered");

    voters[voter].weight = 1;
}

function vote(uint proposal) public {
    Voter storage sender = voters[msg.sender];

    require(sender.weight > 0, "Has no right to vote");
    require(!sender.voted, "Already voted");
    require(proposal < proposals.length, "Invalid proposal");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winner) {
    uint winningVoteCount = 0;

    for (uint i = 0; i < proposals.length; i++) {
      if (proposals[i].voteCount > winningVoteCount) {
        winningVoteCount = proposals[i].voteCount;
        winner = i;
      }
    }
```

```
    }

    function winnerName() public view returns (string memory) {
        return proposals[winningProposal()].name;
    }
}
```

Note: This is a simplified, runnable version illustrating the requested string modification. If your instructor requires the original Remix Ballot.sol, preserve its existing logic and make the equivalent data-type changes rather than replacing the entire contract. This example assumes at least one proposal is provided.

7. PROCEDURE

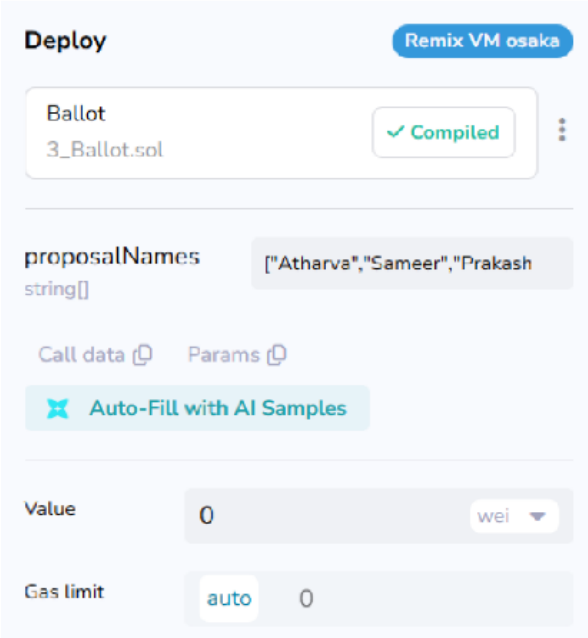
1. Visit https://remix.ethereum.org/.
2. Open the Ballot.sol file from the contracts folder.
3. Understand the Voter and Proposal structures and the functions of the contract.
4. Change the proposal name declaration from bytes32 to string.
5. Update the constructor and other functions wherever the proposal name type is used.
6. Save the file and open the Solidity Compiler tab.
7. Compile the contract and resolve any compilation errors.
8. Open the Deploy & Run Transactions tab.
9. Select Remix VM as the execution environment.
10. Enter proposal names in the constructor field. For example:

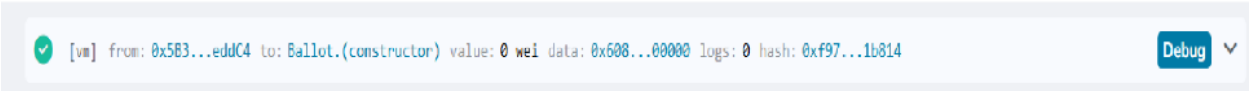
```
["Python", "Solidity", "Java"]
```

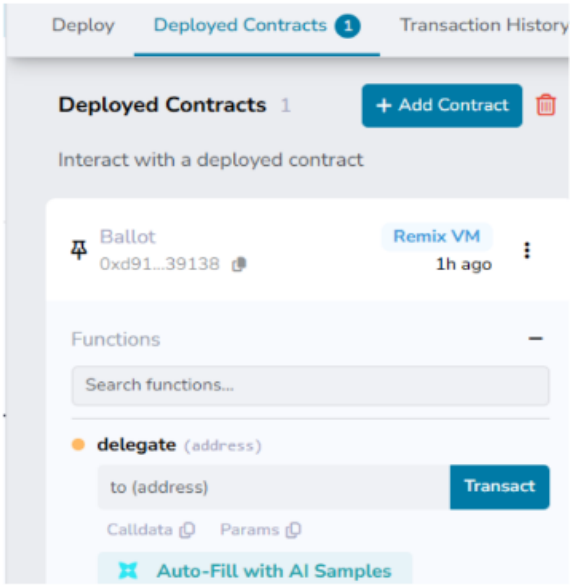
11. Click **Deploy**.
12. Expand the deployed contract under Deployed Contracts.
13. Call proposals(0), proposals(1), and proposals(2) to inspect proposal names and vote counts.
14. Use giveRightToVote to register another account, if necessary.
15. Switch to the registered account, call vote with a valid proposal index, and observe the updated vote count.
16. Call winningProposal() and winnerName() to check the winning proposal.

8. OUTPUT

The contract compiles successfully and is deployed in Remix IDE when the steps above are completed without errors.

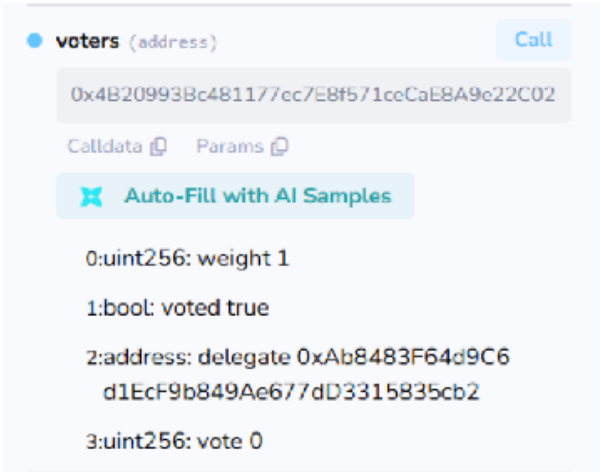

Deploy
Remix VM osaka
Ballot
3_Ballot.sol
Compiled
proposalNames
string[]
["Atharva","Sameer","Prakash
Call data
Params
Auto-Fill with AI Samples
Value
0
wei
Gas limit
auto
0


[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xf97...1b814
Debug

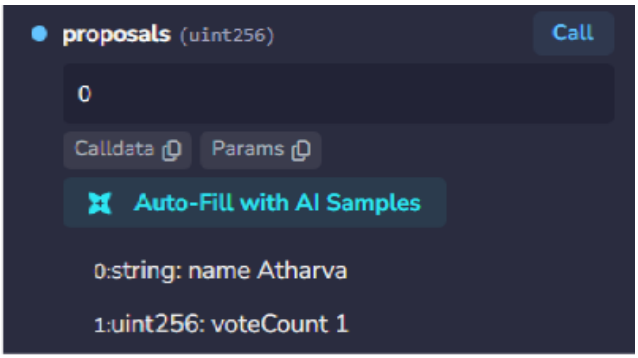

Deploy
Deployed Contracts 1
Transaction History
Deployed Contracts 1
+ Add Contract
Interact with a deployed contract
Ballot
0xd91...39138
Remix VM
1h ago
Functions
Search functions...
delegate (address)
to (address)
Transact
Calldata
Params
Auto-Fill with AI Samples

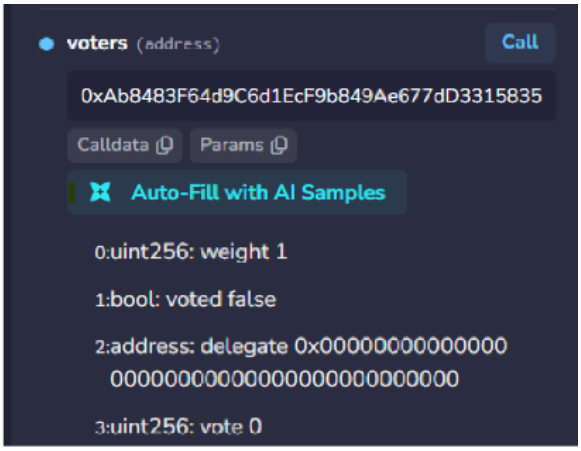

chairperson
Call
0:address: 0x5B38Da6a701c568545dCf
cB03FcB875f56beddC4
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Ballot.chairperson() data: 0x2e4...176cf
Debug

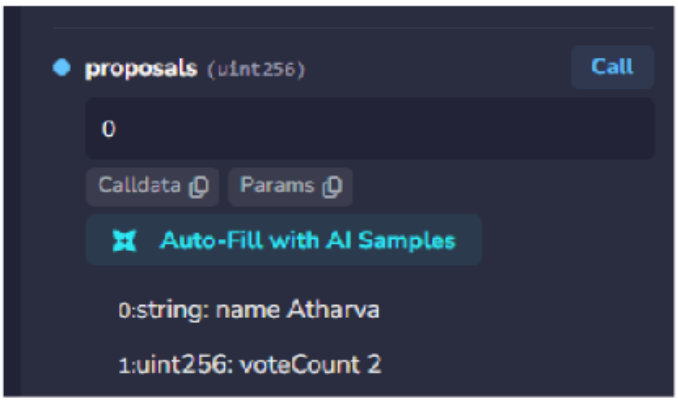

giveRightToVote (address)
Transact
Calldata
Params
[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...c02db logs: 0 hash: 0xf4d...36e0c
Debug

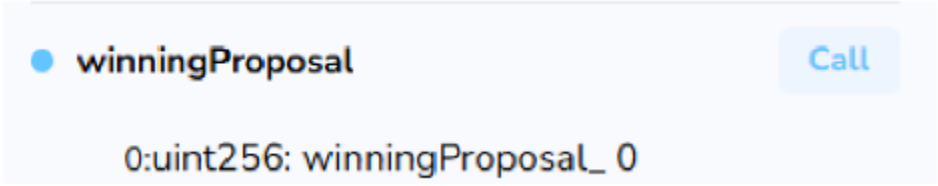

voters (address)
Call
0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02
Calldata
Params
Auto-Fill with AI Samples
0:uint256: weight 1
1:bool: voted true
2:address: delegate 0xAb8483F64d9C6
d1EcF9b849Ae677dD3315835cb2
3:uint256: vote 0

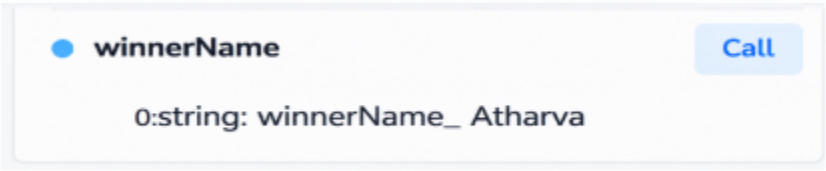

vote (uint256)
Transact
[vm] from: 0x4B8...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00000 logs: 0 hash: 0x03c...29887
Debug


proposals (uint256)
Call
0
Calldata
Params
Auto-Fill with AI Samples
0:string: name Atharva
1:uint256: voteCount 1


voters (address)
Call
0xAb8483F64d9C6d1EcF9b849Ae677dD3315835
Calldata
Params
Auto-Fill with AI Samples
0:uint256: weight 1
1:bool: voted false
2:address: delegate 0x00000000000000
0000000000000000000000000
3:uint256: vote 0







10. CONCLUSION

Thus, the Ballot smart contract was studied and modified to use the string data type for proposal names. The contract's logic, including mappings, storage, memory, access restrictions, and voting operations, was understood. The contract was compiled and deployed using Remix IDE, and its transactions were tested to observe proposal details and voting results.